

Universidad Torcuato Di Tella

Métodos Analíticos y Programación

Prof. Hernán Czemerinski

Clase teórica 2: Algoritmos, programas y condicionales

Algoritmo

Un **algoritmo** es una secuencia finita de instrucciones.

Ejemplo:

1. Moje el cabello
2. Coloque shampoo
3. Masajee suavemente y deje actuar por 2 minutos
4. Enjuague
5. Repita el procedimiento

Algoritmo

Un **algoritmo** es una secuencia finita de instrucciones.

Otro ejemplo:

Ingredientes: 15 huevos, 600 gramos de harina, 600 gramos de azúcar

1. Mientras no estén espumosos, batir los huevos junto con el azúcar
2. Agregar la harina en forma envolvente
3. Batir suavemente
4. Colocar en el horno a 180 grados
5. Si le clavo un cuchillo y sale húmedo, entonces ir a 4
6. Retirar del horno
7. Mientras no esté frío, esperar
8. Desmoldar y servir

Instrucción

Una **instrucción** es una operación que:

- ▶ transforma el *estado* de las cosas; o bien
 - ▶ **modifica el flujo de ejecución**
-
1. Moje el cabello
 2. Coloque shampoo
 3. Masajee suavemente y deje actuar por 2 minutos
 4. Enjuague
 5. **Repita el procedimiento**

Instrucción

Una **instrucción** es una operación que:

- ▶ transforma el *estado* de las cosas; o bien
 - ▶ modifica el flujo de ejecución
1. Mientras no estén espumosos, batir los huevos junto con el azúcar
 2. Agregar la harina en forma envolvente
 3. Batir suavemente
 4. Colocar en el horno a 180 grados
 5. Si le clavo un cuchillo y sale húmedo, entonces ir a 4
 6. Retirar del horno
 7. Mientras no esté frío, esperar
 8. Desmoldar y servir

Programa

Un **programa** es la implementación de uno o varios algoritmos en un lenguaje de programación usando las instrucciones que este provee.

```
demo.py x
1
2 def bubble_sort(list):
3     sorted_list = list[:]
4     is_sorted = False
5     while is_sorted == False:
6         swaps = 0
7         for i in range(len(list) - 1):
8             if sorted_list[i] > sorted_list[i + 1]:
9                 # swap
10                temp = sorted_list[i]
11                sorted_list[i] = sorted_list[i + 1]
12                sorted_list[i + 1] = temp
13                swaps += 1
14            print(swaps)
15            if swaps == 0:
16                is_sorted = True
17        return sorted_list
18
```

Memoria

Durante la ejecución de un programa, sus datos se almacenan en la **memoria**.

La memoria de una computadora es una secuencia numerada de **celdas** en las cuales podemos almacenar datos. La unidad elemental es el **bit**, que toma valores 0 ó 1.

. . .

1	0	1	1	0	0	0	1	0	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---

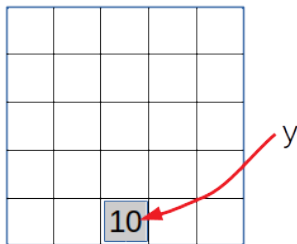
 . . .

Habitualmente, para referirnos a su contenido o a su dimensión se utilizan otras unidades de medida:

- ▶ 8 bits = 1 **byte** (unidad mínima más usada)
- ▶ 1024 bytes = 1 KB (kilobyte)
- ▶ 1024 KB = 1 MB (megabyte)
- ▶ 1024 MB = 1 GB (gigabyte)
- ▶ 1024 GB = 1 TB (terabyte)
- ▶ 1024 TB = 1 PB (petabyte)
- ▶ ...

Variable

En un programa, una **variable** es un nombre que denota una posición de la memoria, en la cual se almacena un valor.



Por simplicidad, decimos que “una variable vale n ”, donde n se reemplaza por un valor. En el ejemplo, que y vale 10.

Variables

Reglas sintácticas de Python para los **nombres de variables**:

1. Solo puede contener caracteres alfanuméricos y guiones bajos (A-Z, a-z, 0-9, _)
2. Tiene que empezar con una letra o un guión bajo
3. Son *case sensitive* (edad, Edad y EDAD son tres variables distintas)

nrotelefono	✓
nro_telefono	✓
_nro_telefono	✓
nroTelefono	✓
nrotelefono2	✓

2nro_telefono	✗
nro-telefono	✗
nro.telefono	✗
nro telefono	✗

Asignación

Una **asignación** es una ecuación orientada que permite almacenar en la posición de memoria denotada por una variable el valor resultante de evaluar una **expresión**.

(*Ayudamemoria:* una expresión es una combinación de valores, variables, operadores y llamados a funciones.)

Sintaxis: **variable = expresión**

Ejemplos:

► `x = 1000`

► `y = x`

► `x = x + y * 10`

► `x = x`

Estado

Se denomina **estado** al valor de todas las variables de un programa en un punto de su ejecución. Es una “foto” de la memoria en un momento determinado.

Ejemplo:

Programa

```
⇒ y = 10  
  x = y * 2  
  y = y + 1
```

Memoria

```
x → ?  
y → ?
```

Estado

Se denomina **estado** al valor de todas las variables de un programa en un punto de su ejecución. Es una “foto” de la memoria en un momento determinado.

Ejemplo:

Programa

```
⇒ y = 10  
  x = y * 2  
  y = y + 1
```

Memoria

```
x → ?  
y → 10
```

Estado

Se denomina **estado** al valor de todas las variables de un programa en un punto de su ejecución. Es una “foto” de la memoria en un momento determinado.

Ejemplo:

Programa

```
y = 10  
x = y * 2  
⇒ y = y + 1
```

Memoria

```
x → 20  
y → 10
```

Estado

Se denomina **estado** al valor de todas las variables de un programa en un punto de su ejecución. Es una “foto” de la memoria en un momento determinado.

Ejemplo:

Programa

```
y = 10  
x = y * 2  
y = y + 1
```



Memoria

```
x → 20  
y → 11
```

Estado

Ejercicio 1. ¿Cuál es el valor de las variables x , y , z luego de la ejecución del siguiente programa?

Programa

```
⇒ x = 5  
  y = x  
  x = 3  
  z = x + y
```

Memoria

```
x → ?  
y → ?  
z → ?
```

Sentencias condicionales



Ejemplo: Reglas del juego Piedra, papel o tijera

- ▶ Si un jugador elige piedra y el otro papel, entonces gana el que elige papel
- ▶ Si un jugador elige papel y el otro tijera, entonces gana el que elige tijera
- ▶ ...

Si **condición** entonces **algo**

Tipo de datos bool

Para representar condiciones que pueden ser verdaderas o falsas en los lenguajes de programación existe el tipo `bool`.

Hay dos valores de verdad posibles: `verdadero` (True) y `falso` (False).

Operaciones: `negación` (not), `conjunción` (and) y `disyunción` (or).

Estas operaciones se definen mediante **tablas de verdad**:

p	not p
True	False
False	True

p	q	p and q	p or q
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

Ejercicios 2. ¿Qué valor devuelve cada una de estas expresiones? (¡Pensar!)

- a) `len('hola')>1 and 1+1==3`
- b) `len('hola')>1 or 1+1==3`
- c) `len('hola')>1 and not (1+1==3)`

Sentencias condicionales

Las **sentencias condicionales** permiten diferentes comportamientos dependiendo del estado del programa durante su ejecución.

```
if <expr>:  
    <sentencias>  
...
```

- ▶ <expr> es una expresión booleana, llamada **guarda**
- ▶ <sentencias> es una serie de instrucciones llamada **cuerpo**
- ▶ El cuerpo se ejecuta si y solo si la guarda es **verdadera**
- ▶ Si la guarda es **falsa**, se ejecuta la instrucción inmediatamente posterior al cuerpo del if

Ejemplo:

```
x = int(input('Ingrese un número: '))  
if x > 5:  
    print('x es mayor que 5.')  
print('Esto no forma parte del cuerpo del if.')
```

Sentencias condicionales

Las **sentencias condicionales** permiten diferentes comportamientos dependiendo del estado del programa durante su ejecución.

```
if <expr>:  
    <sentencias1>  
else:  
    <sentencias2>  
...
```

- ▶ <sentencias₁> se ejecuta si y solo si <expr> es **verdadera**
- ▶ <sentencias₂> se ejecuta si y solo si <expr> es **falsa**

Ejemplo:

```
x = int(input('Ingrese un número: '))  
if x > 5:  
    print('x es mayor que 5.')  
else:  
    print('x es menor o igual que 5.')
```

Sentencias condicionales

Las **sentencias condicionales** permiten diferentes comportamientos dependiendo del estado del programa durante su ejecución.

```
if <expr1>:  
    <sentencias1>  
elif <expr2>:  
    <sentencias2>  
...  
else:  
    <sentenciasn>
```

- ▶ <sentencias₁> se ejecuta si y solo si <expr₁> es **verdadera**
- ▶ <sentencias₂> se ejecuta si y solo si <expr₁> es **falsa** y <expr₂> es **verdadera**
- ▶ ...
- ▶ <sentencias_n> se ejecuta si y solo si todas las expresiones previas son **falsas**

Ejemplo:

```
x = int(input('Ingrese un número: '))  
if x > 5:  
    print('x es mayor que 5.')  
elif x == 5:  
    print('x es igual a 5.')  
else:  
    print('x es menor que 5.')
```

Sentencias condicionales

Reglas sintácticas de Python:

```
if x > y:
    x = x + 1
    y = y - 1
elif x == y:
    ...
else:
    ...
```

- ▶ A continuación de la guarda se escribe el carácter dos puntos ':'
- ▶ Los bloques de código están delimitados por tabulación o una cantidad fija de espacios
- ▶ Tanto los `elif` como el `else` son optativos
- ▶ Se pueden usar tantos `elif`s como se quiera

Sentencias condicionales

Ejercicio 3. ¿Qué imprime el siguiente programa por la pantalla?

Programa

```
⇒ x = 1234  
  y = 987  
  if x > y:  
      z = x  
  else:  
      z = y  
  print(z)
```

Memoria

```
x → ?  
y → ?  
z → ?
```

Consola



Repaso de la clase de hoy

- ▶ Algoritmos y programas
- ▶ Tipo de datos bool:
 - ▶ Valores **True**, **False**.
 - ▶ Operaciones not, and, or.
- ▶ Condicionales: if/elif/else

Bibliografía complementaria:

- ▶ IPP, secciones 5.2 a 5.7.

Con lo visto, ya pueden resolver toda la sección 2 de la guía de ejercicios.